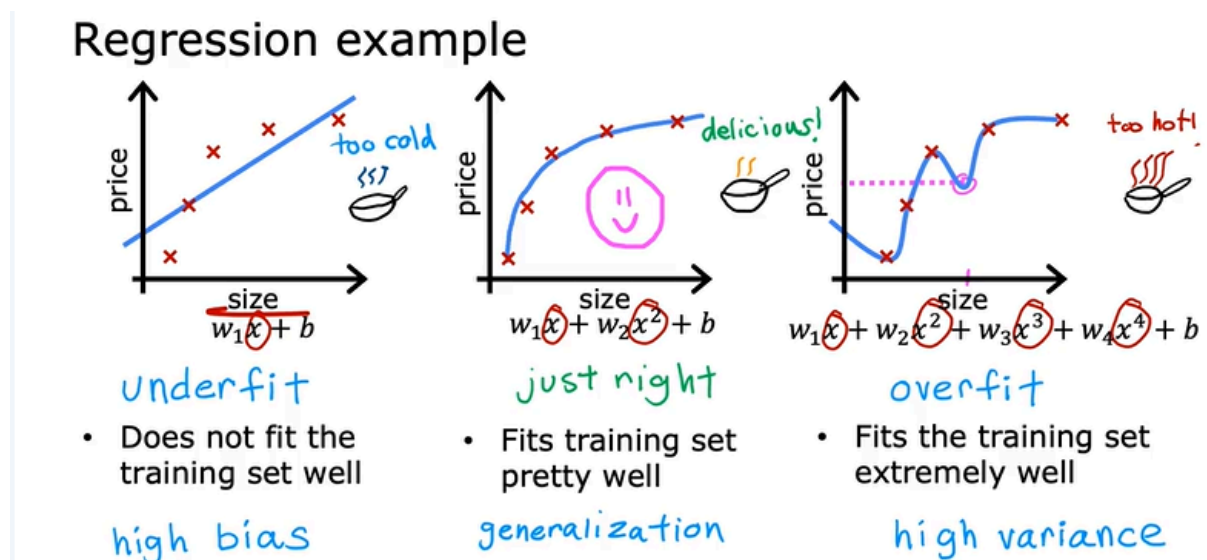


# The problem of overfitting

## The problem of overfitting

You've seen a couple of different learning algorithms, linear regression and logistic regression. They work well for many tasks. But sometimes in an application, the algorithm can run into a problem called overfitting, which can cause it to perform poorly. What I like to do in this video is to show you what is overfitting, as well as a closely-related, almost opposite problem called underfitting.



- Suppose your data-set looks like this, with the input feature  $x$  being the size of the house, and the value,  $y$  that you're trying to predict the price of the house. One thing you could do is fit a linear function to this data. If you do that, you get a straight line fit to the data that maybe looks like this. But this isn't a very good model. Looking at the data, it seems pretty clear that as the size of the house increases, the housing process flattened out.
  - This algorithm does not fit the training data very well. The technical term for this is the model is underfitting the training data. Another term is the algorithm has high bias.
  - You may have read in the news about some learning algorithms really, unfortunately, demonstrating bias against certain ethnicities or certain genders. In machine learning, the term bias has multiple meanings. Checking learning algorithms for bias based on characteristics such as

gender or ethnicity is absolutely critical. But the term bias has a second technical meaning as well, which is the one I'm using here, which is if the algorithm has underfit the data, meaning that it's just not even able to fit the training set that well.

- There's a clear pattern in the training data that the algorithm is just unable to capture. Another way to think of this form of bias is as if the learning algorithm has a very strong preconception, or we say a very strong bias, that the housing prices are going to be a completely linear function of the size despite data to the contrary. This preconception that the data is linear causes it to fit a straight line that fits the data poorly, leading it to [underfitted data](#).
- Now, let's look at a second variation of a model, which is if you insert for a quadratic function at the data with two features,  $x$  and  $x^2$ , then when you fit the parameters  $W_1$  and  $W_2$ , you can get a curve that fits the data somewhat better. Maybe it looks like this (the curve in the middle). Also, if you were to get a new house, that's not in this set of five training examples.
  - This model would probably do quite well on that new house. If you're real estate agents, the idea that you want your learning algorithm to do well, even on examples that are not on the training set, that's called generalization. Technically we say that you want your learning algorithm to generalize well, which means to make good predictions even on brand new examples that it has never seen before. These quadratic models seem to fit the training set not perfectly, but pretty well. I think it would generalize well to new examples.
- Now let's look at the other extreme. What if you were to fit a fourth-order polynomial to the data? You have  $x$ ,  $x^2$ ,  $x^3$ , and  $x^4$  all as features. With this fourth for the polynomial, you can actually fit the curve that passes through all five of the training examples exactly. You might get a curve that looks like this (the curve in the right).
  - This, on one hand, seems to do an extremely good job fitting the training data because it passes through all of the training data perfectly. In fact, you'd be able to choose parameters that will result in the cost function being exactly equal to zero because the errors are zero on all five training examples. But this is a very wiggly curve, its going up and down all over the place. If you have this whole size right here, the model would predict that this house is cheaper than houses that are smaller

than it. We don't think that this is a particularly good model for predicting housing prices.

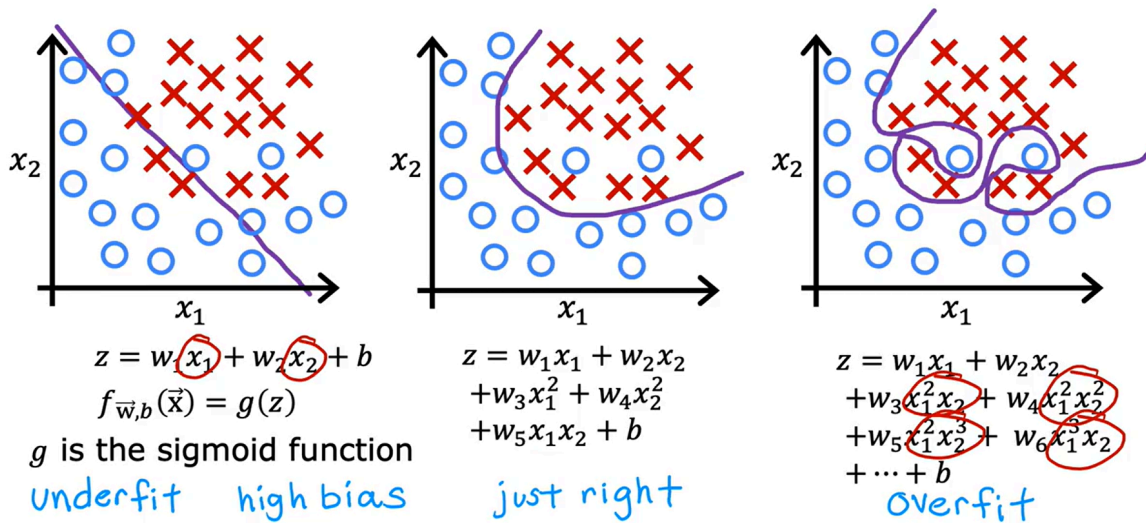
- The technical term is that we'll say this model has overfit the data, or this model has an overfitting problem. Because even though it fits the training set very well, it has fit the data almost too well, hence is overfit. It does not look like this model will generalize to new examples that's never seen before. Another term for this is that the algorithm has high variance.

In machine learning, many people will use the terms over-fit and high-variance almost interchangeably. We'll use the terms underfit and high bias almost interchangeably. The intuition behind overfitting or high-variance is that the algorithm is trying very hard to fit every single training example. It turns out that if your training set were just even a little bit different, say one house was priced just a little bit more or a little bit less, then the function that the algorithm fits could end up being totally different.

If two different machine learning engineers were to fit this fourth-order polynomial model, to just slightly different datasets, they couldn't end up with totally different predictions or highly variable predictions. That's why we say the algorithm has high variance.

Contrasting this rightmost model with the one in the middle for the same house, it seems, the middle model gives them much more reasonable prediction for price. There isn't really a name for this case in the middle, but I'm just going to call this **just right**, because it is neither underfit nor overfit. You can say that the goal machine learning is to find a model that hopefully is neither underfitting nor overfitting. In other words, hopefully, a model that has neither high bias nor high variance. When I think about underfitting and overfitting, high bias and high variance.

# Classification

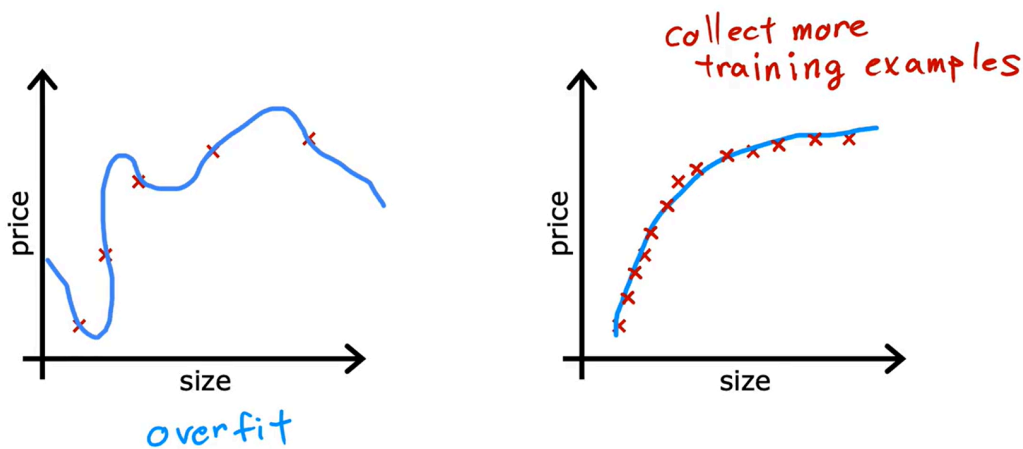


## Addressing overfitting

### Collecting more training examples

Here's our overfit house price prediction model. One way to address this problem is to collect more training data, that's one option. If you're able to get more data, that is more training examples on sizes and prices of houses, then with the larger training set, the learning algorithm will learn to fit a function that is less wiggly. You can continue to fit a high order polynomial or some of the function with a lot of features, and if you have enough training examples, it will still do okay. To summarize, the number one tool you can use against overfitting is to get more training data.

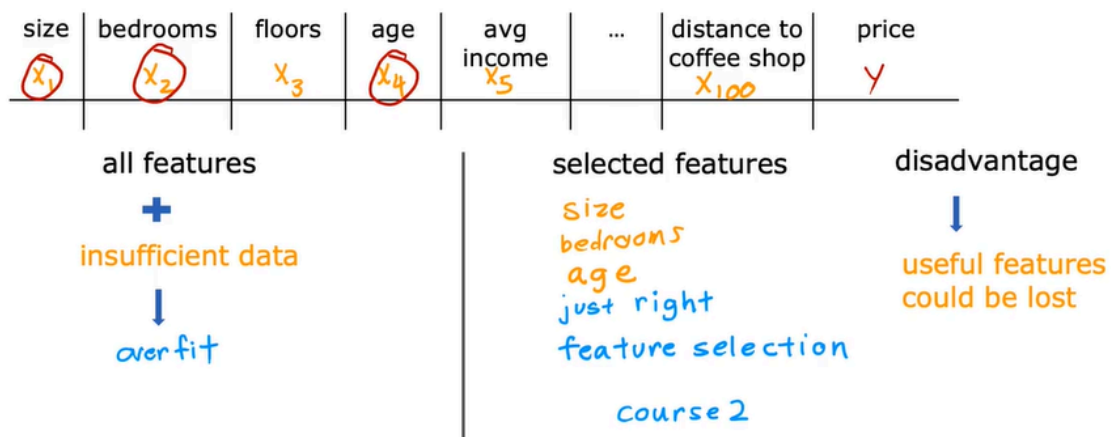
## Collect more training examples



## Selecting features to include/exclude

A second option for addressing overfitting is to see if you can use fewer features.

## Select features to include/exclude



In the previous video, our models features included the size  $x$ , as well as the size squared, and this  $x$  squared, and  $x$  cubed and  $x^4$  and so on. These were a lot of polynomial features. In that case, one way to reduce overfitting is to just not use so many of these polynomial features. But now let's look at a different example.

- Maybe you have a lot of different features of a house of which to try to predict its price, ranging from the size, number of bedrooms, number of

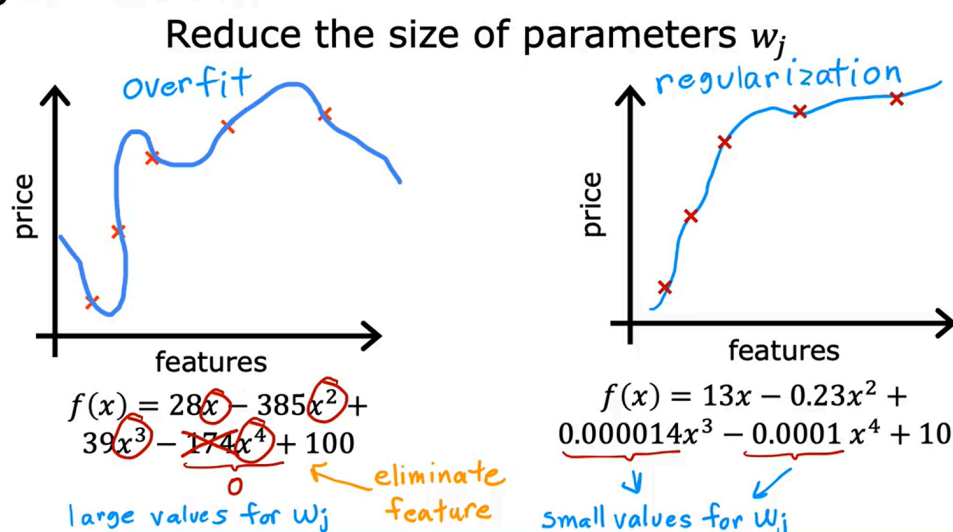
floors, the age, average income of the neighborhood, and so on and so forth, total distance to the nearest coffee shop. It turns out that if you have a lot of features like these but don't have enough training data, then your learning algorithm may also overfit to your training set.

- Now instead of using all 100 features, if we were to pick just a subset of the most useful ones, maybe size, bedrooms, and the age of the house. If you think those are the most relevant features, then using just that smallest subset of features, you may find that your model no longer overfits as badly. Choosing the most appropriate set of features to use is sometimes also called feature selection. One way you could do so is to use your intuition to choose what you think is the best set of features, what's most relevant for predicting the price.
- Now, one disadvantage of feature selection is that by using only a subset of the features, the algorithm is throwing away some of the information that you have about the houses. For example, maybe all of these features, all 100 of them are actually useful for predicting the price of a house. Maybe you don't want to throw away some of the information by throwing away some of the features.

## Regularization

Now, this takes us to the third option for reducing overfitting. This technique, which we'll look at in even greater depth in the next video is called regularization.

## Regularization



This technique, which we'll look at in even greater depth in the next video is called regularization. If you look at an overfit model, here's a model using polynomial features:  $x, x^2, x^3, \dots$ . You find that the parameters are often relatively large. Now if you were to eliminate some of these features, say, if you were to eliminate the feature  $x^4$ , that corresponds to setting this parameter to 0. So setting a parameter to 0 is equivalent to eliminating a feature, which is what we saw on the previous slide.

It turns out that regularization is a way to more gently reduce the impacts of some of the features without doing something as harsh as eliminating it outright. What regularization does is encourage the learning algorithm to shrink the values of the parameters without necessarily demanding that the parameter is set to exactly 0.

It turns out that even if you fit a higher order polynomial like this, so long as you can get the algorithm to use smaller parameter values:  $w_1, w_2, w_3, w_4$ . You end up with a curve that ends up fitting the training data much better. So what regularization does, is it lets you keep all of your features, but they just prevents the features from having an overly large effect, which is what sometimes can cause overfitting. By the way, by convention, we normally just reduce the size of the  $w_j$  parameters, that is  $w_1$  through  $w_n$ . It doesn't make a huge difference whether you regularize the parameter  $b$  as well, you could do so if you want or not if you don't.

## Recap

# Addressing overfitting

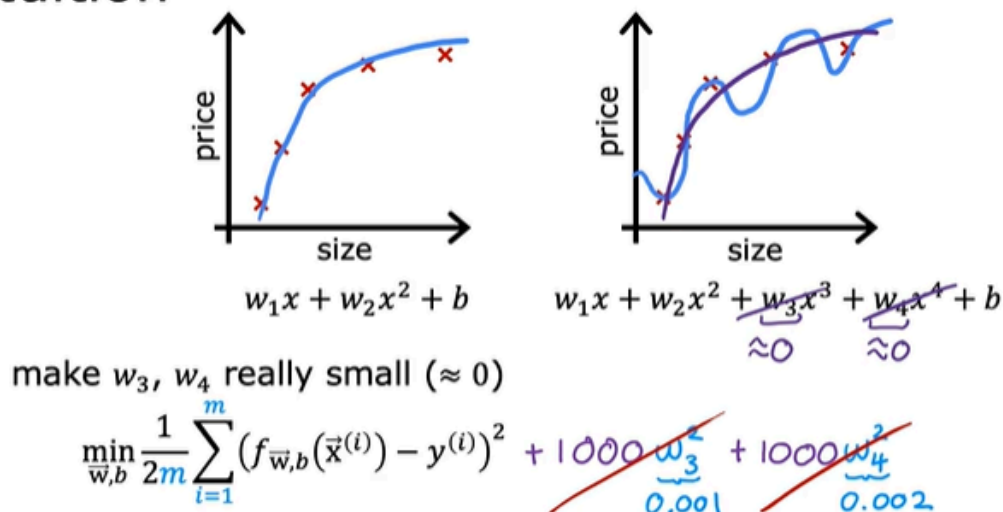
## Options

1. Collect more data
2. Select features
  - Feature selection *in course 2*
3. Reduce size of parameters
  - "Regularization" *next videos!*



# Cost function with regularization

## Intuition



Suppose that you had a way to make the parameters  $w_3$  and  $w_4$  really, really small. Say close to 0. Here's what I mean. Let's say instead of minimizing this objective function, this is a cost function for linear regression. Let's say you were to modify the cost function and add to it 1000 times  $w_3$  squared plus 1000 times  $w_4$  squared. And here I'm just choosing 1000 because it's a big number but any other really large number would be okay.

So with this modified cost function, you could in fact be penalizing the model if  $w_3$  and  $w_4$  are large. Because if you want to minimize this function, the only way to make this new cost function small is if  $w_3$  and  $w_4$  are both small, right? Because otherwise this 1000 times  $w_3$  squared and 1000 times  $w_4$  square terms are going to be really, really big. So when you minimize this function, you're going to end up with  $w_3$  close to 0 and  $w_4$  close to 0. So we're effectively nearly canceling out the effects of the features  $x^3$  and  $x^4$  and getting rid of these two terms over here. And if we do that, then we end up with a fit to the data that's much closer to the quadratic function, including maybe just tiny contributions from the features  $x^3$  and  $x^4$ . And this is good because it's a much better fit to the data compared to if all the parameters could be large and you end up with this weekly quadratic function



## Regularization

|                                        |                   |                 |              |                     |     |                                      |                        |                 |
|----------------------------------------|-------------------|-----------------|--------------|---------------------|-----|--------------------------------------|------------------------|-----------------|
| small values $w_1, w_2, \dots, w_n, b$ |                   |                 |              |                     |     |                                      | simpler model          | $w_3 \approx 0$ |
|                                        |                   |                 |              |                     |     |                                      | less likely to overfit | $w_4 \approx 0$ |
| size<br>$x_1$                          | bedrooms<br>$x_2$ | floors<br>$x_3$ | age<br>$x_4$ | avg income<br>$x_5$ | ... | distance to coffee shop<br>$x_{100}$ | price<br>$y$           |                 |
| $w_1, w_1, w_2, \dots, w_{100}, b$     |                   |                 |              |                     |     |                                      | $n$ features           | $n = 100$       |

$$J(\vec{w}, b) = \frac{1}{2m} \left[ \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^n w_j^2}_{\text{"lambda" regularization term}} + \underbrace{\frac{\lambda}{2m} b^2}_{\text{can include or exclude } b} \right]$$

regularization parameter  $\lambda > 0$

The idea is that if there are smaller values for the parameters, then that's a bit like having a simpler model. Maybe one with fewer features, which is therefore less prone to overfitting. On the last slide we penalize or we say we regularized only  $w_3$  and  $w_4$ . But more generally, the way that regularization tends to be implemented is if you have a lot of features, say a 100 features, you may not know which are the most important features and which ones to penalize. So the way regularization is typically implemented is to penalize all of the features or more precisely, you penalize all the  $w_j$  parameters and it's possible to show that this will usually result in fitting a smoother simpler, less wacky function that's less prone to overfitting.

So for this example, if you have data with 100 features for each house, it may be hard to pick an advance which features to include and which ones to exclude. So let's build a model that uses all 100 features. So you have these 100 parameters  $w_1$  through  $w_{100}$ , as well as 100 and first parameter  $b$ .

Because we don't know which of these parameters are going to be the important ones. Let's penalize all of them a bit and shrink all of them by adding this new term lambda times the sum from  $j$  equals 1 through  $n$  where  $n$  is 100. The number of features of  $w_j^2$ . This value lambda here is the Greek alphabet lambda  $\lambda$  ( $>0$ ) and it's also called a **regularization parameter**. So similar to picking a learning rate alpha, you now also have to choose a number for lambda.

A couple of things I would like to point out by convention, instead of using lambda times the sum of  $w_j^2$ . We also divide lambda by  $2m$  so that both the 1st and 2nd terms here are scaled by  $1$  over  $2m$ . It turns out that by scaling both

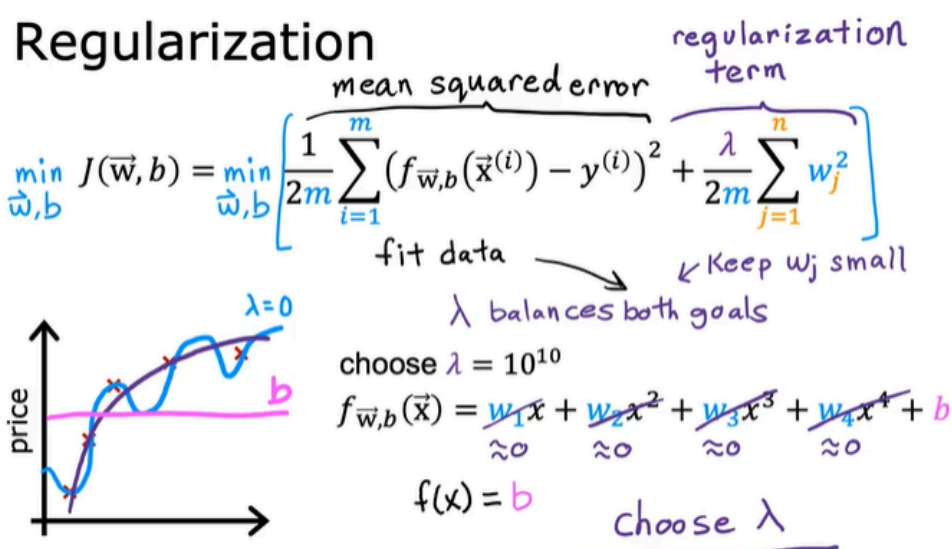
terms the same way it becomes a little bit easier to choose a good value for lambda.

And in particular you find that even if your training set size growth, say you find more training examples. So  $m$  (the training set size) is now bigger. The same value of lambda that you've picked previously is now also more likely to continue to work if you have this extra scaling by  $2m$ . Also by the way, by convention we're not going to penalize the parameter  $b$  for being large. In practice, it makes very little difference whether you do or not. And some machine learning engineers and actually some learning algorithm implementations will also include  $\frac{\lambda}{2m} b^2$ . But this makes very little difference in practice and the more common convention which was used in this course is to regularize only the parameters  $w$  rather than the parameter  $b$ .

## Summarize

if lambda is 0 (the blue curve) this model will **over fit** If lambda is enormous like  $10^{10}$ . This model will **under fit** (the pink curve). And so what you want is some value of lambda that is in between that more appropriately balances these first and second terms of trading off, minimizing the mean squared error and keeping the parameters small.

And when the value of lambda is not too small and not too large, but just right, then hopefully you end up able to fit a 4th order polynomial, keeping all of these features, but with a function that looks like this (the purple curve).



# Regularized linear regression

## Regularized linear regression

$$\min_{\vec{w}, b} J(\vec{w}, b) = \min_{\vec{w}, b} \left[ \frac{1}{2m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2 \right]$$

Gradient descent

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j$$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

} simultaneous update

don't have to regularize b

shrink : thu nhỏ

## Implementing gradient descent

repeat {

$$w_j = w_j - \alpha \left[ \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} w_j \right]$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

} simultaneous update  $j=1 \dots n$

$$w_j = \underbrace{w_j \left( 1 - \alpha \frac{\lambda}{m} \right)}_{\text{shrink } w_j} - \underbrace{\alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}}_{\text{usual update}}$$

$$\alpha \frac{\lambda}{m} = 0.01 \frac{1}{50} = 0.0002$$

$$w_j (1 - 0.0002) = 0.9998 w_j$$

Repeat until convergence:

$$w_j = w_j - \alpha \left[ \frac{1}{m} \sum_{i=1}^m \left( f_{w,b}(x^{(i)}) - y^{(i)} \right) x_j^{(i)} + \frac{\lambda}{m} w_j \right] \quad \text{for } j = 1..n$$

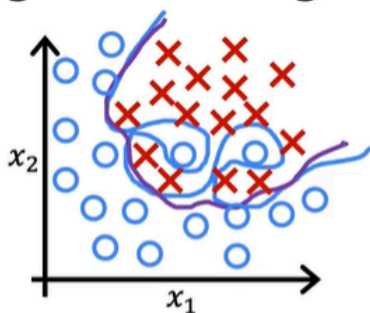
$$b = b - \alpha \frac{1}{m} \sum_{i=0}^{m-1} \left( f_{w,b}(x^{(i)}) - y^{(i)} \right)$$

How we get the derivative term (optional)

$$\begin{aligned} \frac{\partial}{\partial w_j} J(\vec{w}, b) &= \frac{\partial}{\partial w_j} \left[ \frac{1}{2m} \sum_{i=1}^m \left( \underbrace{f(\vec{x}^{(i)})}_{\vec{w} \cdot \vec{x}^{(i)} + b} - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2 \right] \\ &= \frac{1}{2m} \sum_{i=1}^m \left[ (\vec{w} \cdot \vec{x}^{(i)} + b - y^{(i)}) \cancel{2} x_j^{(i)} \right] + \frac{\lambda}{2m} \cancel{2} w_j \quad \text{No } \sum_{j=1}^n \\ &= \frac{1}{m} \sum_{i=1}^m \left[ (\underbrace{\vec{w} \cdot \vec{x}^{(i)} + b}_{f(\vec{x})} - y^{(i)}) x_j^{(i)} \right] + \frac{\lambda}{m} w_j \\ &= \frac{1}{m} \sum_{i=1}^m \left[ (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right] + \frac{\lambda}{m} w_j \end{aligned}$$

## Regularized logistic regression

### Regularized logistic regression



$$z = w_1 x_1 + w_2 x_2 + w_3 x_1^2 x_2 + w_4 x_1^2 x_2^2 + w_5 x_1^2 x_2^3 + \dots + b$$

$$f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{-z}}$$

Cost function

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[ y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

$\min_{\vec{w}, b} J(\vec{w}, b) \rightarrow w_j \downarrow$

# Regularized logistic regression

$$J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[ y^{(i)} \log(f_{\vec{w}, b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

$\min_{\vec{w}, b}$

Gradient descent

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$j=1 \dots n$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

}

Looks same as  
for linear regression!

$$= \frac{1}{m} \sum_{i=1}^m \left[ (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right] + \frac{\lambda}{m} w_j$$

logistic regression

$$= \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

don't have to  
regularize